

Database Schema & Data Model

1. Purpose

The database must support a content-heavy, multilingual, source-backed learning platform.

The schema should make it easy to:

- Store scriptures and verses correctly
- Keep original text separate from translations and commentary
- Support English and Hindi
- Connect concepts, deities, mantras, festivals, and scriptures
- Track sources and verification
- Support user bookmarks and learning progress
- Support AI retrieval and citations
- Expand to more languages and scriptures later

Recommended database:

PostgreSQL

Recommended provider:

Supabase

2. Core Design Principle

Never combine these into one field:

Original Scripture

Translation

Commentary

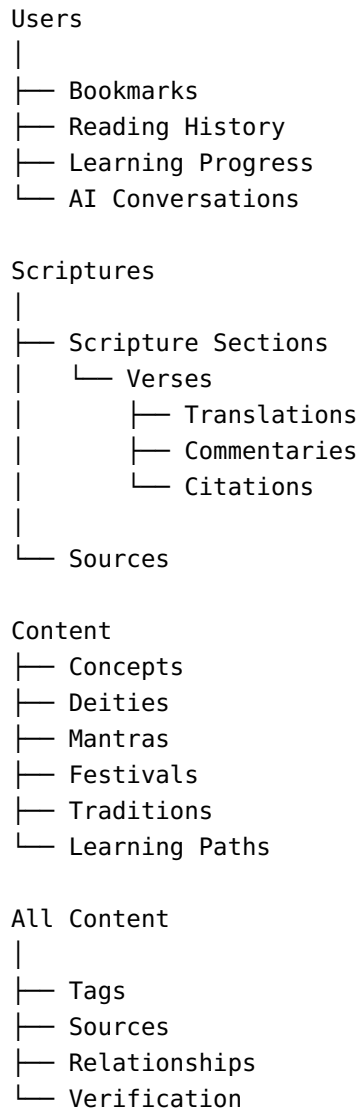
Editorial Explanation

AI Explanation

They should be separate database entities.

This allows the website to show users exactly what type of information they are reading.

3. High-Level Entity Map



4. Users Table

Table:

users

Authentication may be managed by Supabase Auth.

Application profile data should be stored separately.

Fields:

```
id
email
created_at
updated_at
last_login_at
status
```

The authentication provider may own some of these fields.

5. Profiles Table

Table:

profiles

Fields:

```
id
user_id
display_name
avatar_url
preferred_language
preferred_theme
created_at
updated_at
```

Recommended language values:

```
en
hi
```

Future languages can be added later.

6. Roles Table

Table:

```
roles
```

Possible roles:

```
visitor
user
editor
reviewer
admin
```

Fields:

```
id
name
description
```

7. User Roles Table

Table:

```
user_roles
```

Fields:

```
user_id
role_id
assigned_at
assigned_by
```

A user may potentially hold more than one administrative role.

8. Scriptures Table

Table:

scriptures

Purpose:

Stores major scripture-level records.

Examples:

- Bhagavad Gita
- Rig Veda
- Katha Upanishad
- Ramayana
- Mahabharata

Fields:

```
id
slug
canonical_name
sanskrit_name
short_description
scripture_type
primary_language
structure_type
verification_status
created_at
updated_at
published_at
```

Example `scripture_type` values:

```
veda
upanishad
itihasa
purana
gita
sutra
smriti
other
```

9. Scripture Names Table

To support multiple languages:

Table:

`scripture_localizations`

Fields:

```
id
scripture_id
locale
title
description
summary
seo_title
seo_description
```

This avoids duplicating the scripture itself.

10. Scripture Sections Table

Different scriptures have different structures.

For example:

Bhagavad Gita:

Chapter → Verse

Ramayana:

Kanda → Sarga → Verse

Mahabharata:

Parva → Subsection → Verse

Therefore use a flexible hierarchy.

Table:

scripture_sections

Fields:

```
id
scripture_id
parent_section_id
section_type
section_number
sort_order
canonical_title
sanskrit_title
slug
created_at
updated_at
```

Possible `section_type` values:

```
chapter
kanda
sarga
parva
mandala
sukta
adhyaya
pada
book
section
```

11. Scripture Section Localizations

Table:

scripture_section_localizations

Fields:

```
id
section_id
locale
title
```

summary
description

12. Verses Table

Table:

verses

Fields:

```
id
scripture_id
section_id
verse_number
canonical_reference
sanskrit_text
normalized_sanskrit
sort_order
source_edition_id
verification_status
created_at
updated_at
```

Example:

```
canonical_reference:
Bhagavad Gita 2.47
```

Important:

Use a unique database constraint to prevent duplicate verse references within the same scripture and section.

13. Verse Variants

Some texts may contain variant readings.

Table:

verse_variants

Fields:

```
id
verse_id
variant_text
source_id
variant_type
notes
verification_status
```

This can be added when needed.

14. Transliteration Table

Table:

transliterations

Fields:

```
id
verse_id
system
text
source_id
verification_status
```

Recommended initial system:

IAST

Future:

- ISO
 - simplified pronunciation
-

15. Translations Table

Table:

translations

Fields:

```
id
verse_id
locale
translator_id
translation_text
source_id
translation_type
copyright_status
license_id
verification_status
created_at
updated_at
```

Possible translation_type:

```
published
platform
literal
interpretive
```

16. Translators Table

Table:

translators

Fields:

```
id
name
biography
tradition_id
```

```
website
notes
```

A translator may or may not belong to a specific tradition.

17. Commentators Table

Table:

```
commentators
```

Fields:

```
id
name
sanskrit_name
tradition_id
biography
historical_period
notes
```

18. Commentaries Table

Table:

```
commentaries
```

Fields:

```
id
verse_id
commentator_id
locale
commentary_text
summary
source_id
copyright_status
license_id
verification_status
```

```
created_at
updated_at
```

This allows several commentaries for the same verse.

19. Platform Explanations

Table:

```
editorial_explanations
```

Fields:

```
id
content_type
content_id
locale
simple_explanation
deep_explanation
author_id
reviewer_id
verification_status
created_at
updated_at
```

Examples:

- verse explanation
 - concept explanation
 - deity explanation
-

20. Sources Table

This is one of the most important tables.

Table:

```
sources
```

Fields:

```
id
title
source_type
author
editor
translator
publisher
edition
publication_year
language
isbn
url
archive_url
license_id
copyright_status
tradition_id
verification_status
notes
created_at
updated_at
```

Possible `source_type`:

```
scripture
commentary
translation
book
journal
academic_paper
institution
manuscript
website
other
```

21. Licenses Table

Table:

```
licenses
```

Fields:

```
id
name
short_name
url
allows_redistribution
allows_modification
allows_commercial_use
notes
```

Examples:

```
Public Domain
CC BY
CC BY-SA
Custom Permission
All Rights Reserved
```

22. Source Verification Table

Table:

```
source_reviews
```

Fields:

```
id
source_id
reviewer_id
status
review_notes
reviewed_at
```

Possible status:

```
proposed
reviewed
approved
restricted
```

rejected
archived

23. Traditions Table

Table:

traditions

Fields:

id
slug
name
sanskrit_name
parent_tradition_id
description
created_at
updated_at

Examples:

Advaita Vedanta
Vishishtadvaita
Dvaita
Shaivism
Vaishnavism
Shaktism
Smarta

24. Tradition Localizations

Table:

tradition_localizations

Fields:

```
id
tradition_id
locale
name
description
```

25. Concepts Table

Table:

concepts

Examples:

- Dharma
- Karma
- Moksha
- Atman
- Brahman
- Samsara
- Maya
- Bhakti

Fields:

```
id
slug
canonical_name
sanskrit_name
difficulty_level
verification_status
created_at
updated_at
published_at
```

26. Concept Localizations

Table:

concept_localizations

Fields:

```
id
concept_id
locale
title
short_definition
simple_explanation
deep_explanation
seo_title
seo_description
```

27. Deities Table

Table:

deities

Fields:

```
id
slug
canonical_name
sanskrit_name
parent_deity_id
verification_status
created_at
updated_at
published_at
```

A parent relationship can support forms or manifestations where appropriate.

This relationship should be used carefully and reviewed editorially.

28. Deity Localizations

Table:

deity_localizations

Fields:

```
id
deity_id
locale
name
short_description
overview
philosophical_significance
iconography
seo_title
seo_description
```

29. Mantras Table

Table:

mantras

Fields:

```
id
slug
canonical_name
sanskrit_text
mantra_type
primary_deity_id
source_id
verification_status
created_at
updated_at
published_at
```

Possible mantra types:

```
vedic
bija
stotra
shloka
dhyana
nama
```

peace
other

30. Mantra Localizations

Table:

mantra_localizations

Fields:

id
mantra_id
locale
title
transliteration
meaning
context
traditional_usage
pronunciation_notes

31. Festivals Table

Table:

festivals

Fields:

id
slug
canonical_name
sanskrit_name
date_type
verification_status
created_at
updated_at
published_at

Because festival dates change yearly, do not store only one permanent date.

32. Festival Dates Table

Table:

festival_dates

Fields:

```
id
festival_id
year
start_date
end_date
timezone
region
calendar_basis
calculation_source
verification_status
```

This allows regional and yearly differences.

33. Festival Localizations

Table:

festival_localizations

Fields:

```
id
festival_id
locale
name
short_description
meaning
history
practices
spiritual_significance
regional_variations
```

```
seo_title
seo_description
```

34. Tags Table

Table:

```
tags
```

Fields:

```
id
slug
name
tag_type
```

Examples:

```
karma
bhakti
krishna
moksha
meditation
```

35. Content Tags Table

Table:

```
content_tags
```

Fields:

```
id
content_type
content_id
tag_id
```

Example:

Bhagavad Gita 2.47 can be tagged with:

```
karma
karma-yoga
action
attachment
dharma
```

36. Content Relationships

This is critical for creating a knowledge network.

Table:

```
content_relationships
```

Fields:

```
id
from_content_type
from_content_id
relationship_type
to_content_type
to_content_id
created_by
verification_status
```

Examples:

```
Karma
related_to
Bhagavad Gita 2.47
```

```
Krishna
associated_with
Janmashtami
```

```
Maha Shivaratri
associated_with
Shiva
```

37. Relationship Types

Possible values:

```
related_to
explained_by
mentioned_in
associated_with
commented_by
practiced_in
celebrated_as
part_of
supports
contrasts_with
```

These should be controlled values rather than arbitrary text.

38. Learning Paths

Table:

```
learning_paths
```

Fields:

```
id
slug
difficulty_level
estimated_duration
verification_status
created_at
updated_at
published_at
```

Examples:

Sanatan Dharma for Beginners
Introduction to Bhagavad Gita
Understanding Karma Yoga

39. Learning Path Localizations

Table:

`learning_path_localizations`

Fields:

```
id
learning_path_id
locale
title
description
```

40. Learning Modules

Table:

`learning_modules`

Fields:

```
id
learning_path_id
content_type
content_id
position
is_required
```

Example:

Beginner Path:

```
1 What is Sanatan Dharma?  
2 Dharma  
3 Karma  
4 Atman  
5 Brahman  
6 Samsara  
7 Moksha
```

41. User Learning Progress

Table:

```
user_learning_progress
```

Fields:

```
id  
user_id  
learning_path_id  
module_id  
status  
started_at  
completed_at  
last_accessed_at
```

Possible status:

```
not_started  
in_progress  
completed
```

42. Bookmarks

Instead of separate bookmark tables for every content type, use a generic model.

Table:

```
bookmarks
```

Fields:

```
id
user_id
content_type
content_id
created_at
```

Supported content types:

```
verse
concept
scripture
deity
mantra
festival
article
```

Unique constraint:

```
user_id + content_type + content_id
```

This prevents duplicate bookmarks.

43. Reading History

Table:

```
reading_history
```

Fields:

```
id
user_id
content_type
content_id
first_viewed_at
last_viewed_at
view_count
```

This supports:

- Continue Reading
- Recently Viewed
- Recommendations

44. AI Conversations

Table:

`ai_conversations`

Fields:

```
id
user_id
title
locale
created_at
updated_at
deleted_at
```

Anonymous conversations do not necessarily need long-term storage.

45. AI Messages

Table:

`ai_messages`

Fields:

```
id
conversation_id
role
message_text
model
created_at
```

Possible roles:

```
user
assistant
system
```

System prompts may not need to be stored permanently for every conversation.

46. AI Citations

Table:

```
ai_message_citations
```

Fields:

```
id
message_id
source_id
content_type
content_id
chunk_id
citation_order
```

This allows AI answers to show exactly which sources were used.

47. AI Retrieval Chunks

Table:

```
knowledge_chunks
```

Fields:

```
id
content_type
content_id
source_id
locale
chunk_text
chunk_index
```

```
verification_status
embedding
metadata
created_at
updated_at
```

The `embedding` field can use:

pgvector

Only approved content should enter the production retrieval index.

48. Knowledge Chunk Metadata

Useful metadata:

```
scripture_id
chapter
verse
tradition_id
commentator_id
language
content_type
verification_status
source_id
```

Metadata filtering is important during AI retrieval.

49. Search Aliases

Table:

```
search_aliases
```

Fields:

```
id
content_type
content_id
locale
```

```
alias
normalized_alias
```

Example for Krishna:

```
Krishna
Kṛṣṇa
कृष्ण
Krushna
```

Search aliases help users find content despite spelling variation.

50. Search Documents

A materialized search table or generated index may eventually be useful.

Table:

```
search_documents
```

Possible fields:

```
id
content_type
content_id
locale
title
search_text
keywords
search_vector
popularity_score
updated_at
```

This can improve performance and ranking.

51. Content Review Workflow

Table:

```
content_reviews
```

Fields:

```
id
content_type
content_id
reviewer_id
review_type
status
notes
created_at
completed_at
```

Possible review types:

```
editorial
scripture
sanskrit
translation
tradition
legal
source
```

52. Content Verification Status

Use controlled values:

```
draft
source_check
under_review
verified
published
needs_review
archived
```

Not every table needs all statuses, but the terminology should remain consistent.

53. Content Revision History

Table:

`content_revisions`

Fields:

```
id
content_type
content_id
revision_number
changed_by
change_summary
snapshot
created_at
```

The `snapshot` can store JSON of the previous state.

This makes it possible to restore old content.

54. Error Reports

Table:

`content_reports`

Fields:

```
id
user_id
content_type
content_id
report_type
description
status
assigned_to
created_at
resolved_at
```

Possible report types:

```
incorrect_sanskrit
incorrect_translation
wrong_source
misleading_explanation
```

```
typo
tradition_issue
copyright
other
```

55. Media Assets

Table:

```
media_assets
```

Fields:

```
id
storage_path
media_type
title
alt_text
copyright_status
license_id
source_id
uploaded_by
created_at
```

Possible types:

```
image
audio
pdf
document
```

56. Content Media Relationships

Table:

```
content_media
```

Fields:

```
id
content_type
content_id
media_id
usage_type
position
```

Usage types:

```
hero
thumbnail
gallery
audio
source_document
```

57. Notifications

Future table:

```
notifications
```

Fields:

```
id
user_id
type
title
message
link
read_at
created_at
```

Not required for initial MVP.

58. User Preferences

Table:

```
user_preferences
```

Fields:

```
user_id
daily_shloka_enabled
festival_notifications_enabled
learning_reminders_enabled
default_translation_id
default_commentary_id
preferred_reading_mode
updated_at
```

Many of these features can remain inactive until later versions.

59. App Configuration

Table:

app_settings

For administrative settings.

Possible fields:

```
key
value
description
updated_by
updated_at
```

Examples:

```
anonymous_ai_daily_limit
registered_ai_daily_limit
maintenance_mode
default_locale
```

Sensitive secrets should NOT be stored here.

60. AI Usage Tracking

Table:

ai_usage

Fields:

```
id
user_id
session_id
request_type
model
input_tokens
output_tokens
estimated_cost
success
created_at
```

Useful for controlling AI costs.

Avoid storing unnecessary private message content here.

61. Admin Audit Log

Table:

audit_logs

Fields:

```
id
user_id
action
entity_type
entity_id
old_values
new_values
ip_address
created_at
```

Important actions:

- Publishing
- Deleting
- Source changes
- Role changes
- Verification changes

62. Main Scripture Relationship Example

Bhagavad Gita:

```
scriptures
├── Bhagavad Gita
│   ├── scripture_sections
│   │   ├── Chapter 1
│   │   ├── Chapter 2
│   │   └── ...
│   └── verses
│       └── 2.47
│           ├── transliterations
│           ├── translations
│           ├── commentaries
│           ├── editorial_explanations
│           ├── content_tags
│           └── source citations
```

63. Example Verse Record

Conceptual example:

Verse

ID:

verse_bg_2_47

Scripture:

Bhagavad Gita

Chapter:

2

Verse:

47

Reference:

Bhagavad Gita 2.47

Sanskrit:

[verified Sanskrit]

Verification:

Verified

Translations:

English Translation A

English Translation B

Hindi Translation A

Commentaries:

Commentary A

Tradition: Advaita

Commentary B

Tradition: Vishishtadvaita

Commentary C

Tradition: Dvaita

All remain separate.

64. Example Concept Relationship

Concept:

Karma

Relationships:

Karma

→ related_to → Dharma

Karma

→ related_to → Samsara

Karma

→ related_to → Moksha

Karma

→ mentioned_in → Bhagavad Gita 2.47

Karma

→ mentioned_in → Bhagavad Gita Chapter 3

This is how the knowledge network will be built.

65. Database Constraints

Important constraints should include:

- Unique scripture slug
- Unique concept slug
- Unique deity slug
- Unique mantra slug
- Unique festival slug
- Unique verse within a scripture section
- Unique bookmark per user/content
- Valid foreign keys
- Valid language codes
- Valid verification statuses

Database constraints should prevent common content mistakes.

66. Delete Strategy

Avoid permanent deletion of important content.

Use:

```
deleted_at  
archived_at
```

for critical records when possible.

Scriptures and sources should normally be archived instead of deleted.

67. Referential Integrity

Do not allow deleting:

- A scripture with active verses
- A source referenced by translations
- A commentator with active commentaries

unless dependencies are handled intentionally.

Use restrictive foreign-key behavior for important knowledge data.

68. Row-Level Security

RLS should protect user-specific data.

Users should only access their own:

- Bookmarks
- Reading history
- Learning progress
- AI conversations
- Preferences

Editors/reviewers need separate administrative authorization.

69. Public Data

Public read access can be permitted for published records such as:

- Scriptures

- Published verses
- Published concepts
- Published deity pages
- Published mantras
- Published festivals

Drafts should never be publicly accessible.

70. Admin Data Access

Administrative writes should occur through secure server-side operations.

Do not expose administrator-level database credentials to the browser.

71. Database Indexes

Important indexes:

```
scriptures.slug
concepts.slug
deities.slug
mantras.slug
festivals.slug

verses.scripture_id
verses.section_id

translations.verse_id
commentaries.verse_id

content_tags.tag_id

bookmarks.user_id

reading_history.user_id

knowledge_chunks.content_id
```

Search fields should also receive appropriate full-text and vector indexes.

72. Vector Index

For `knowledge_chunks.embedding`:

Use pgvector indexes once the knowledge base becomes sufficiently large.

The exact vector index type should be decided based on scale and query performance.

73. Database Backup Requirements

Production database should have:

- Automated backups
- Point-in-time recovery if affordable
- Periodic exports of critical scripture data

Source metadata and verified text should also have offline/version-controlled backups.

74. Seed Data

Development should include seed scripts.

Initial seed data:

- User roles
- Languages
- Verification statuses
- Source types
- Relationship types
- Scripture categories
- Initial Bhagavad Gita structure

Do not manually recreate these values on every environment.

75. Migration Requirements

Use database migrations for every schema change.

Example:

```
001_initial_schema
002_add_scriptures
003_add_translations
004_add_ai_chunks
```

Migrations should be committed to Git.

76. V1 Database Scope

Must exist in Version 1:

```
profiles

scriptures
scripture_sections
verses
transliterations
translations

sources

concepts
concept_localizations

deities
deity_localizations

mantras
mantra_localizations

festivals
festival_localizations
festival_dates

tags
content_tags
content_relationships

learning_paths
learning_modules
user_learning_progress

bookmarks
```

```
reading_history

ai_conversations
ai_messages
ai_message_citations
knowledge_chunks

content_reviews
content_reports
```

Some administrative/support tables can be added as implementation progresses.

77. Recommended Schema Evolution

Phase 1

Core users and content.

Phase 2

Scripture and translations.

Phase 3

Relationships and learning.

Phase 4

Search.

Phase 5

AI/RAG.

Phase 6

Advanced content governance.

This prevents building the entire schema before any feature exists.

78. Scalability Principle

The database should support future additions such as:

- More languages
- More scriptures
- More commentaries
- Audio
- Native apps
- Temples
- Panchang
- Courses
- Community

However, these should not complicate the initial implementation unnecessarily.

79. Database Design Rule

The most important rule is:

Store meaning and provenance, not just text.

For every important religious statement, the system should eventually be able to answer:

- What is this?
 - Where did it come from?
 - Who translated it?
 - Who interpreted it?
 - Which tradition does it represent?
 - Has it been verified?
 - Can the AI use it?
 - Can the user view the source?
-

80. Status

Database Schema & Data Model:

Initial Version Complete

Core architecture now covers:

- Users

- Scriptures
- Sections
- Verses
- Translations
- Commentaries
- Sources
- Traditions
- Concepts
- Deities
- Mantras
- Festivals
- Learning
- Search
- Bookmarks
- AI retrieval
- Citations
- Verification
- Audit and correction workflows

Next document:

AI Architecture & RAG Specification